PROJECT REPORT

Ultimate AI Agent: Integrating Rule-Based Personal Assistance and Chatbot

Name: Shivansh Dayal

Registration Number: 25BCE11239

Course: Fundamentals in AI and ML CSA2001

INTRODUCTION:

This agent is an offline, Command Line Interface (CLI) application that enables users to interact with it through text-based commands. This agent is developed as a rule-based chatbot system, in which users' input is processed through regular expression patterns to determine the users' intentions and respond appropriately.

This system is an extension of the basic chatbot system that enables users to perform various tasks through the integration of additional functionalities. This system helps fill the gap between personal productivity tools and support systems, as well as basic operating system functionalities. The system is developed as an extension of Symbolic Artificial Intelligence and natural language pattern-matching concepts, which enables users to interact with it in a more intuitive manner. This enables users to perform various tasks, including daily tasks, mathematical calculations, recording expenses, and accessing academic-related information.

MOTIVATION OF THE PROJECT:

The problem of extreme digital fragmentation or divide is faced by many students. A student uses one app for task management, another for calculators, spreadsheets for budgeting, and struggles through various university web portals for academic purposes. In addition, to interact with desktop applications, students are forced to remove their hands from the keyboard to use the mouse. The idea behind creating this project is to provide a single localized AI solution to fulfill personal utility, financial management, system automation, and academic purposes in one seamless terminal interface, which operates instantaneously and completely offline.

PROBLEM STATEMENT:

There is a practical need for an instantaneous text-based query system with an understanding of conversational English and multi-domain utility. There is a need for students to have an instantaneous way of logging their expenses, running required student applications, and

getting precise academic answers without needing any help from human sources, heavy web browsers, or invasive cloud-based AI.

OBJECTIVE OF THE PROJECT:

To create a full CLI-based application that effectively applies concepts learned throughout the course, namely Condition-Based Logic, goal oriented, and Text Pattern-Matching, to determine user intent, automate operating system-level commands, process financial data, and store personal records locally using plain text files.

EXISTING METHODS:

Currently, people make use of highly segregated systems for personal tasks like voice assistants, which have privacy issues and latency problems; budgeting applications, which require GUI navigation; and static web pages for university-related questions.

PROS AND CONS OF THE STATED METHODS:

PROS:

- **Cloud-Based AI:** It is highly versatile, able to perform a wide variety of unconstrained tasks.
- **Dedicated GUI Budgeting Apps:** They are visually appealing with an intuitive graphical user interface.

CONS:

- **Cloud-Based AI:** It demands huge computational power, needs to be connected to the internet all the time, and compromises the privacy of the user by collecting their data.

- **Dedicated GUI Budgeting Apps:** It faces the problem of high digital friction, which means that it takes a long time to start, and the user needs to use their mouse to log in a single, simple purchase.

HARDWARE AND SOFTWARE REQUIREMENTS:

- **Hardware:** Any computer/ laptop, capable of running the code in a command terminal.

- **Software:** Python 3.x.

  The project uses built-in basic modules (re, datetime, random, os), demonstrating that complex system automation and AI logic do not require heavy external libraries.

METHODOLOGY AND GOAL:

This project uses Symbolic AI.

The process is explained below:

The process is to create a "Hierarchical Intent Tree." When the user enters a string, the system normalizes the string and passes it through a hierarchy of regex-based tests. It tries to find system automation commands (launching apps), mathematical expressions, expense logging, and hardcoded utilities (Tasks). Finally, the text is checked against unique Knowledge Bases for it to function as a chatbot (Academic and IT Support). The process aims at accurately matching unconstrained human text to the correct domain execution.

FUNCTIONAL MODULES DESIGN AND ANALYSIS:

1. **Input Processor:** Normalizes text (lowercase, stripping whitespace) for analysis by AI.

2. **System Automation Module:** Understands natural language to interpret OS commands (e.g., clearing screen, launching Calculator/Notepad apps).
3. **Financial Tracking Module:** Extracts financial information from user conversations, logging to user_expenses.txt.
4. **Task & Memory Module:** Reads/writes user task lists to user_tasks.txt.
5. **Mathematical Engine:** Sanitizes extracted strings to remove alphabetical characters, safely executing equations using Python's built-in eval() function.
6. **Expert System Knowledge Bases:** Dictionaries of IT troubleshooting rules for natural language conversations, Academic policies.

ALGORITHM DEVELOPMENT:

The system's intelligence relies on a Hierarchical Intent Router, which utilizes Symbolic AI and semantic pattern-matching rather than neural networks. The algorithmic flow is divided into three distinct processing methodologies:

Sequential Pattern Matching and Extraction: The core intent-matching algorithm relies on direct structural analysis. It uses sequential scanning to find specific text patterns. For the Financial Tracking module, the algorithm searches for structural patterns like spent [number] on [item]. In a single pass, it isolates two distinct variables—the numerical amount and the string description—and passes them directly to the file-handling function to update the persistent ledger.

Natural Language Math Translation and Zero-Trigger Routing: The mathematical algorithm handles true natural language through a multi-step pre-processing chain. Before sanitization, it translates human vocabulary (e.g., "multiplied by", "ans") into Python-readable operators and stored state variables. To enable seamless conversational math, the algorithm uses a pattern-matching process of elimination. The AI is also able to solve the maths problems without needing a keyword for it to initialise the process.

Bounded System Execution Logic: In the context of OS-level automation, the algorithm avoids the use of dynamic wildcard variables to prevent Command Injection attacks. Instead, the algorithm utilizes a bounded routing approach. The algorithm checks the user input for

specific hardware-safe keywords such as 'notepad' or 'calculator.' Once the keyword is found in the user input, the algorithm executes the code asynchronously with the os.system("start calc") function, providing complete cybersecurity while successfully integrating the Python environment with the host OS.

SOFTWARE ARCHITECTURAL DIAGRAM:

1. **User Layer**

   - User interacts through CLI (Command Line Interface)
   - Provides text input
   - Receives responses or system actions

2. **Core Processing Layer (UltimateAssistant)**

   2.1 Command Interpreter

   - Analyzes user input
   - Identifies intent (task, math, expense, etc.)
   - Routes request to appropriate module

   2.2 Task Manager

   - Adds new tasks
   - Displays task list
   - Saves tasks to file

   2.3 Math Engine

   - Processes mathematical expressions
   - Converts words into operators (e.g., plus → +)
   - Evaluates expressions using eval()
   - Stores last result for reuse

   2.4 Expense Tracker

- Records expenses (amount + item)
- Retrieves expense history
- Maintains a simple ledger

2.5 Academic & IT Helper

- Uses predefined regex patterns
- Provides responses for:
  - Academic queries (fees, exams, grades)
  - Basic IT issues

3. **<u>Data Storage Layer</u>**

- user_tasks.txt → Stores task list
- user_expenses.txt → Stores expense records
- logs.txt → Stores chat history

4. **<u>System Interaction Layer</u>**

- Opens applications:
  - Notepad / Text Editor
  - Calculator
- Performs system operations:
  - Fetch current time
  - Clear terminal screen

5. **<u>Flow Summary</u>**

1. User enters input
2. Input is processed by Command Interpreter
3. Request is routed to relevant module
4. Module performs action
5. Data is stored/retrieved if needed
6. Response is returned to user

<u>Architecture Type:</u> Modular functional separation inside a single class.

USER → Command Handler → Main Assistant → (Tasks | Math | Expenses | Academic | IT) → Storage → System Actions

CODING:

The software is built using Python. It deals with input validation elegantly , strictly adhering to the "Complete CLI Application" guidelines, which include complex user interaction, core functionalities in various domains and local data storage.

```
import re
import datetime
import random
import os
import sys

class UltimateAssistant:
    def __init__(self):
        self.task_file = "user_tasks.txt"
        self.log_file = "logs.txt"
        self.expense_file = "user_expenses.txt"
        self.last_math_result = 0
        self.tasks = []
        self.load_tasks()

        self.academic_help = {
            r'(fee|tuition|pay|cost|dues)': [
                "Tuition and hostel fees can be paid using the student portal.",
                "Fee payment deadlines for this semester are already provided in
email. Check your email for dates."
            ],
            r'(course|register|enroll|credits)': [
                "Course registration opens two weeks before the semester starts.",
                "Advisor approval is needed for core courses."
            ],
            r'(grade|gpa|cgpa|marks|result)': [
                "Results are on the portal. Minimum CGPA is 6.0.",
                "Contact your professor for grade discrepancies within 48 hours."
            ],
            r'(exam|schedule|timetable|date)': [
                "Final exam schedules are released one month prior to exams."
            ],
            r'(password|login|access|account)': [
                "Use 'Forgot Password' with your university email to reset access."
            ]
        }
```

```
        self.it = {
            r'.*my (.*) is not working.*': "Have you tried restarting your {0} or
checking its connection?",
            r'.*screen is (.*).*': "If your screen is {0}, try checking the display
cable.",
            r'hello|hi': "Hello! I am your AI Assistant. How can I help?"
        }

    def load_tasks(self):
        if os.path.exists(self.task_file):
            with open(self.task_file, 'r') as f:
                self.tasks = [line.strip() for line in f.readlines()]

    def save_tasks(self):
        with open(self.task_file, 'w') as f:
            for task in self.tasks:
                f.write(task + "\n")

    def log_chat(self, user_given_input, bot_out):
        with open(self.log_file, 'a') as f:
            timestamp = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")
            f.write(f"[{timestamp}] USER: {user_given_input} | AI: {bot_out}\n")

    def calculate_math(self, expression):
        expression = expression.replace("ans", str(self.last_math_result))
        expression = expression.replace("previous", str(self.last_math_result))

        expression = expression.replace("plus", "+").replace("add", "+")
        expression = expression.replace("minus", "-").replace("subtract", "-")
        expression = expression.replace("times", "*").replace("multiplied by",
"*").replace("multiply", "*")
        expression = expression.replace("divided by", "/").replace("divide",
"/").replace("over", "/")

        cleaned_expression = re.sub(r'[^0-9+\-*/().]', '', expression)
        try:
            if not cleaned_expression:
                return "No valid numbers found in your request."

            result = eval(cleaned_expression)
            self.last_math_result = result
            return str(result)
        except:
            return "Mathematical error. Please ensure the equation is valid."

    def save_expense(self, amount, item):
        with open(self.expense_file, 'a') as f:
            f.write(f"{datetime.datetime.now().ctime()} | Amount: {amount} | Item:
{item}\n")
```

```
    def read_expenses(self):
        if not os.path.exists(self.expense_file):
            return "No expenses logged yet."
        with open(self.expense_file, 'r') as f:
            return "Your Expense Ledger:\n" + f.read()

    def process(self, user_input):
        text = user_input.lower().strip()

        if "time" in text:
            return datetime.datetime.now().strftime("It is currently %I:%M %p.")

        task_match = re.search(r'remind me to (.*)|add task (.*)', text)
        if task_match:
            t = task_match.group(1) if task_match.group(1) else task_match.group(2)
            self.tasks.append(t.strip())
            self.save_tasks()
            return f"Task '{t.strip()}' added to your planner."

        if "show tasks" in text or "my list" in text:
            if not self.tasks:
                return "Your task list is empty."
            return "Your Tasks:\n" + "\n".join([f"{i+1}. {x}" for i, x in
enumerate(self.tasks)])

        math_match = re.search(r'calculate (.*)|what is (.*)|solve (.*)', text)
        if math_match:
            expression = next(g for g in math_match.groups() if g is not None)
            return f"The result is: {self.calculate_math(expression)}"

        for pattern, responses in self.academic_help.items():
            if re.search(pattern, text):
                return random.choice(responses)

        for pattern, response in self.it.items():
            match = re.search(pattern, text)
            if match:
                if '{0}' in response and match.groups():
                    return response.format(match.group(1))
                return response

        temp = re.sub(r'ans|previous|plus|minus|times|divided
by|divide|over|add|subtract|multiplied by', '', text)
        leftovers = re.sub(r'[0-9\s\+\-\*\/\(\)]', '', temp)
        if leftovers == "" and len(text) > 0:
            return f"The result is: {self.calculate_math(text)}"

        expense_match = re.search(r'spent ([0-9]+) on (.*)|cost ([0-9]+) for (.*)',
text)
        if expense_match:
```

```
            amt = expense_match.group(1) if expense_match.group(1) else
expense_match.group(3)
            item = expense_match.group(2) if expense_match.group(2) else
expense_match.group(4)
            self.save_expense(amt, item.strip())
            return f"Logged expense: {amt} for {item.strip()}."

        if "show expenses" in text or "my ledger" in text:
            return self.read_expenses()

        sys_match = re.search(r'open (.*)|launch (.*)', text)
        if sys_match:
            app = next(g for g in sys_match.groups() if g is not None).strip()

            if "notepad" in app or "text" in app:
                if os.name == 'nt':
                    os.system("start notepad")
                elif sys.platform == "darwin":
                    os.system("open -a TextEdit")
                return "Launched Text Editor."

            elif "calculator" in app or "calc" in app:
                if os.name == 'nt':
                    os.system("start calc")
                elif sys.platform == "darwin":
                    os.system("open -a Calculator")
                return "Launched Calculator."

            return f"Cannot launch '{app}'. App not recognized or OS not
supported."

        if "clear screen" in text or "cleaned_expression terminal" in text:
            os.system('cls' if os.name == 'nt' else 'clear')
            return "Terminal cleared."

        return "I am an intro-level AI. I do not recognize that command or
question."

    def run(self):
        print("================================================")
        print("      AI Assistant & Academic Helpdesk    ")
        print("================================================")
        print("Capabilities: Task Management | Math | IT Support | Academic
Queries")
        print("Type 'exit' to quit and save logs.\n")

        while True:
            try:
                user_given_input = input("You: ")
                if not user_given_input:
                    continue
```

```python
                if user_given_input.lower() in ['exit', 'quit', 'stop','bye']:
                    print("AI: Saving data. Shutting down.")
                    break

                out = self.process(user_given_input)
                print(f"AI: {out}\n")
                self.log_chat(user_given_input, out)

            except KeyboardInterrupt:
                break

if __name__ == "__main__":
    UltimateAssistant().run()
```

OUTPUT:

```
==============================================
      AI Assistant & Academic Helpdesk
==============================================
Capabilities: Task Management | Math | IT Support | Academic Queries
Type 'exit' to quit and save logs.

You: hi
AI: Hello! I am your AI Assistant. How can I help?

You: Remind me to submit the BYOP assignment
AI: Task 'submit the byop assignment' added to your planner.

You: Add task buy groceries
AI: Task 'buy groceries' added to your planner.

You: Show my list
AI: Your Tasks:
1. submit the byop assignment
2. buy groceries

You: 20*2 and 20 plus 2
AI: I am an intro-level AI. I do not recognize that command or question.

You: 20*2
AI: The result is: 40

You: My screen is flickering
AI: If your screen is flickering, try checking the display cable.

You: When does the exam schedule come out?
AI: Final exam schedules are released one month prior to exams.

You: I forgot my portal login password
AI: Use 'Forgot Password' with your university email to reset access.

You: What is the time right now?
AI: It is currently 02:37 AM.
```

```
You: i spent 450 on lunch
AI: Logged expense: 450 for lunch.

You: spent 340 on dinner
AI: Logged expense: 340 for dinner.

You: show expense
AI: I am an intro-level AI. I do not recognize that command or question.

You: show expenses
AI: Your Expense Ledger:
Tue Mar 31 02:39:18 2026 | Amount: 450 | Item: lunch
Tue Mar 31 02:39:30 2026 | Amount: 340 | Item: dinner


You: open texteditor
AI: Launched Text Editor.

You: open calculator
AI: Launched Calculator.

You: bye
AI: Saving data. Shutting down.
```

KEY IMPLEMENTATIONS OUTLINE OF THE SYSTEM:

- **Dual-Variable Regex Extraction:** The expense module was successful in extracting two different variables from a single unconstrained sentence, i.e., the integer value and the string description, within a single pass.
- **Cross-Environment OS Routing:** The system utilizes the os.system() built-in module, which helps bridge the gap between the Python terminal environment and the host computer's graphical operating system.

SIGNIFICANT PROJECT OUTCOMES:

It demonstrates that one simple text interface can replace many graphical applications and the addition of expense tracking and OS-level application launching makes the AI system achieve a status of more than just a simple "chatbot" and becoming a true productivity operating system.

TESTING AND REFINEMENT:

- **Challenge:** Launching system applications via CLI commands is strictly dependent on the host Operating System. A command that works on Windows will crash on Mac/Linux.
- **Refinement:** I implemented the python module sys which checks the software on which the system is running so as to function better and execute the right command that is written.
- **Challenge:** During a natural conversation testing, it was discovered that we do not always use mathematical symbols (like * or /) and often want to chain calculations together (e.g., “What is 10 times 10”, “Add 9 and 9”, etc.). The strict coding was making this task difficult to be achieved.
- **Refinement:** I introduced a translation step inside the **calculate_math** function. The AI now translates "times" to * and replaces the variables "ans" or "previous" with the stored memory of a previous output. This successfully bridged the gap between conversational English and rigid symbolic math.

PROJECT APPLICABILITY ON REAL-WORLD APPLICATIONS:

This system is actually quite similar to tools used in real-world system administration. Many engineers/employees use command-line tools that understand simple natural language, so they can run scripts, clear logs, or check system status directly from the terminal without switching between different applications. It just makes their workflow faster and more convenient.

FINDINGS/LEARNINGS OF THE PROJECT:

I learned that strict pattern-matching is incredibly powerful for organizing data. By setting up specific text rules, we can easily pull structured information—like a financial ledger or ask the date time—straight out of casual, messy English conversation. This proves that we don't always need complex neural networks to understand natural language.

CONSTRAINTS OF THE SYSTEM:

The main drawback is the static nature of the system integration. This is because the AI system is only aware of how to launch the applications that are integrated into its logic block (Calculator and Notepad). It is unable to look through the computer's hard drive and launch third-party applications.

CONCLUSION:

The project successfully demonstrates how rule-based AI techniques can be applied to build a practical and efficient CLI-based assistant. It integrates multiple functionalities into a single system and highlights the effectiveness of symbolic AI in solving real-world problems.

FUTURE ENHANCEMENTS:

The enhancements for the future would include adding a data visualization component. The AI would be able to read the user's expenses.txt file, summarize the numerical data, and display a bar chart in the terminal using ASCII characters to display user spending trends for the month. Also I would like to expand the Financial Tracking module's pattern-matching dictionary to recognize income trigger words (like 'earned' or 'received') to calculate earnings too, rather than just tracking expenditures.

REFERENCES:

1. **Python Official Documentation:** https://docs.python.org/3/
2. **Regular Expressions Guide:** https://www.w3schools.com/python/python_regex.asp
3. **Course Materials & Lectures:** Fundamentals of AI and ML. (Referenced for the academic framework, problem statement formulation, and core concepts regarding offline, localized Artificial Intelligence execution).